

SVM in R

Kachun Huang

2025-04-22

SVM in R

Import Library

```
library(tidyverse)
```

```
## Warning: package 'purrr' was built under R version 4.3.3
```

```
## Warning: package 'lubridate' was built under R version 4.3.3
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
```

```
## v dplyr      1.1.4      v readr      2.1.5
```

```
## v forcats    1.0.0      v stringr    1.5.1
```

```
## v ggplot2    3.5.1      v tibble     3.2.1
```

```
## v lubridate  1.9.4      v tidyr      1.3.1
```

```
## v purrr      1.0.4
```

```
## -- Conflicts ----- tidyverse_conflicts() --
```

```
## x dplyr::filter() masks stats::filter()
```

```
## x dplyr::lag()     masks stats::lag()
```

```
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(caret)
```

```
## Warning: package 'caret' was built under R version 4.3.3
```

```
## Loading required package: lattice
```

```
## Warning: package 'lattice' was built under R version 4.3.3
```

```
##
```

```
## Attaching package: 'caret'
```

```
##
```

```
## The following object is masked from 'package:purrr':
```

```
##
```

```
## lift
```

```
library(kernlab)
```

```
## Warning: package 'kernlab' was built under R version 4.3.3
```

```
##
```

```
## Attaching package: 'kernlab'
```

```
##
```

```
## The following object is masked from 'package:purrr':
```

```
##
```

```
##     cross
```

```
##
```

```
## The following object is masked from 'package:ggplot2':
```

```
##
```

```
##     alpha
```

```
library(e1071)
```

```
## Warning: package 'e1071' was built under R version 4.3.3
```

Step 1: Data Pre-processing

Feature Scaling (Standardization or Normalization)

- SVM is **sensitive to the scale** of features — especially with **RBF**, **polynomial**, and **linear kernels**.

Handle Categorical Variables

- SVM requires **numerical input only**
- Convert categorical features using:
 - **One-hot encoding** (preferred)
 - **Label encoding** (only if the categories have ordinal meaning)

Remove or Impute Missing Values

- SVM does not handle NA or NaN values.
- You must either:
 - Remove rows with missing values
 - Impute them (mean, median, mode, or model-based imputation)

Step 2: Model Fitting

```
model <- train(  
  response~.,  
  data = ...,  
  method = "svmLinear",  
  trControl = trainControl("cv", number = 10),  
  preProcess = c("center", "scale") #This will scale all the predictor in the model  
)
```

Step 3: Prediction

```
predicted.classes <- model %>% predict(test.data)
```

Step 4: Confusion Matrix

```
confusionMatrix(  
  factor(predicted.classes),  
  factor(data$response),  
  positive = "1"  
)
```

Hyper-parameter Tuning

We can utilize `tuneGrid` and `tuneLength` parameters in `train()` function.

Tune C parameter

```
model <- train(  
  response~.,  
  data = ...,  
  method = "svmLinear",  
  trControl = trainControl("cv", number = 10),  
  tuneGrid = expand.grid(C = seq(0,2,length = 10))  
)
```

- This creates 20 evenly spaced values between 0 and 2.
- These are the values of the **regularization parameter C** the model will try.

TuneLength

`caret` will automatically create a grid of **different C values** (on a log scale by default), and test them via cross-validation.

```
model <- train(  
  response~.,  
  data = ...,  
  method = "svmLinear",  
  trControl = trainControl("cv", number = 10),  
  tuneLength = n  
)
```

Example

Import Data

```
data <- read.csv("Titanic.csv")
```

Data Pre-Processing

Remove Columns

```
data <- subset(  
  data,  
  select = -c(PassengerId, Name, Ticket, Cabin)  
)
```

Remove Missing Value in Age Column

```
data <- subset(data, is.na(Age) == FALSE)
```

Factorize Categorical Variable

```
data$Pclass <- as.factor(data$Pclass)  
data$Survived <- as.factor(data$Survived)
```

Standardize Numeric Column

```
data$Age <- scale(data$Age)  
data$Fare <- scale(data$Fare)
```

Data Splitting

```
set.seed(123)  
training.samples <- data$Survived %>%  
  createDataPartition(p = 0.75, list = FALSE)  
train.data <- data[training.samples, ]  
test.data <- data[-training.samples, ]  
str(train.data)
```

```
## 'data.frame':   536 obs. of  8 variables:  
## $ Survived: Factor w/ 2 levels "0","1": 2 2 1 1 2 2 2 1 1 2 ...  
## $ Pclass  : Factor w/ 3 levels "1","2","3": 1 3 3 3 2 3 1 3 3 2 ...  
## $ Sex     : chr  "female" "female" "male" "male" ...
```

```
## $ Age      : num [1:536, 1] 0.571 -0.255 0.365 -1.907 -1.081 ...
## $ SibSp    : int  1 0 0 3 1 1 0 0 0 0 ...
## $ Parch    : int  0 0 0 1 0 1 0 0 0 0 ...
## $ Fare     : num [1:536, 1] 0.6914 -0.5059 -0.5035 -0.2574 -0.0874 ...
## $ Embarked: chr   "C" "S" "S" "S" ...
```

```
str(test.data)
```

```
## 'data.frame': 178 obs. of 8 variables:
## $ Survived: Factor w/ 2 levels "0","1": 1 2 1 2 1 1 2 1 1 1 ...
## $ Pclass : Factor w/ 3 levels "1","2","3": 3 1 1 3 3 3 2 1 1 3 ...
## $ Sex     : chr   "male" "female" "male" "female" ...
## $ Age     : num [1:178, 1] -0.53 0.365 1.673 -0.186 0.64 ...
## $ SibSp   : int  1 1 0 0 1 1 0 3 1 0 ...
## $ Parch   : int  0 0 0 2 5 0 0 2 0 0 ...
## $ Fare    : num [1:178, 1] -0.5186 0.3478 0.3244 -0.4452 -0.0646 ...
## $ Embarked: chr   "S" "S" "S" "S" ...
```

Model Fitting

```
set.seed(123)
model <- train(
  Survived ~., data = train.data, method = "svmLinear",
  trControl = trainControl("cv", number = 10),
  # preProcess = c("center", "scale")
  # This will scale all the predictors in the model
)
```

Prediction

```
predicted.classes <- model %>% predict(test.data)
```

Confusion Matrix

```
confusionMatrix(
  factor(predicted.classes),
  factor(test.data$Survived),
  positive = '1'
)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction 0  1
##           0 93 23
##           1 13 49
##
```

```
##           Accuracy : 0.7978
##           95% CI : (0.7312, 0.8541)
##      No Information Rate : 0.5955
##      P-Value [Acc > NIR] : 7.457e-09
##
##           Kappa : 0.5706
##
##  McNemar's Test P-Value : 0.1336
##
##           Sensitivity : 0.6806
##           Specificity : 0.8774
##      Pos Pred Value : 0.7903
##      Neg Pred Value : 0.8017
##           Prevalence : 0.4045
##      Detection Rate : 0.2753
##      Detection Prevalence : 0.3483
##      Balanced Accuracy : 0.7790
##
##      'Positive' Class : 1
##
```

Hyper-parameter Tuning

```
with tuneGrid = expand.grid(C = seq(0, 2, length = 20))
```

```
set.seed(123)
model <- train(
  Survived ~., data = train.data, method = "svmLinear",
  trControl = trainControl("cv", number = 10),
  # preProcess = c("center", "scale")
  # This will scale all the predictors in the model
  tuneGrid = expand.grid(C = seq(0, 2, length = 20))
)
```

```
## Warning: model fit failed for Fold01: C=0.0000 Error in .local(x, ...) :
##   No Support Vectors found. You may want to change your parameters
```

```
## Warning: model fit failed for Fold02: C=0.0000 Error in .local(x, ...) :
##   No Support Vectors found. You may want to change your parameters
```

```
## Warning: model fit failed for Fold03: C=0.0000 Error in .local(x, ...) :
##   No Support Vectors found. You may want to change your parameters
```

```
## Warning: model fit failed for Fold04: C=0.0000 Error in .local(x, ...) :
##   No Support Vectors found. You may want to change your parameters
```

```
## Warning: model fit failed for Fold05: C=0.0000 Error in .local(x, ...) :
##   No Support Vectors found. You may want to change your parameters
```

```
## Warning: model fit failed for Fold06: C=0.0000 Error in .local(x, ...) :
##   No Support Vectors found. You may want to change your parameters
```

```
## Warning: model fit failed for Fold07: C=0.0000 Error in .local(x, ...) :
##   No Support Vectors found. You may want to change your parameters

## Warning: model fit failed for Fold08: C=0.0000 Error in .local(x, ...) :
##   No Support Vectors found. You may want to change your parameters

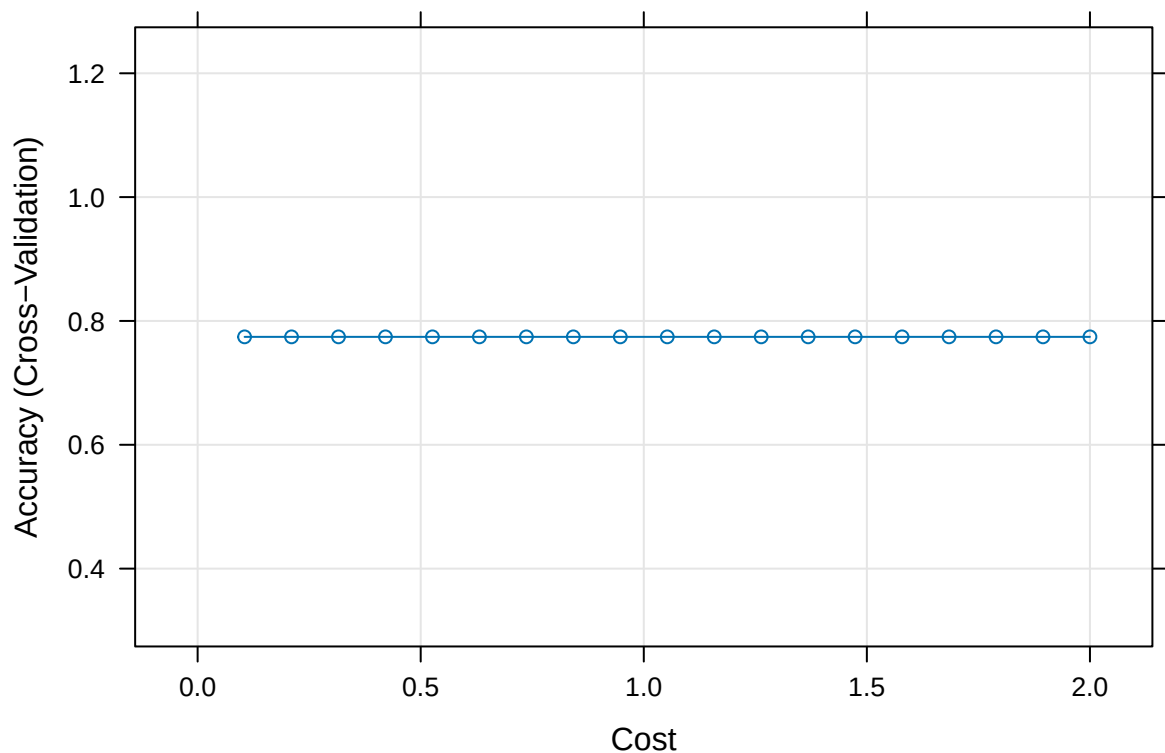
## Warning: model fit failed for Fold09: C=0.0000 Error in .local(x, ...) :
##   No Support Vectors found. You may want to change your parameters

## Warning: model fit failed for Fold10: C=0.0000 Error in .local(x, ...) :
##   No Support Vectors found. You may want to change your parameters

## Warning in nominalTrainWorkflow(x = x, y = y, wts = weights, info = trainInfo,
## : There were missing values in resampled performance measures.

## Warning in train.default(x, y, weights = w, ...): missing values found in
## aggregated results
```

```
plot(model)
```



Plot

```
model$bestTune
```

Best Tune

```
##           C
## 2 0.1052632
```

Here you can directly use the fitted model or you can refit a model using the best tuning parameter.

```
set.seed(123)
model <- train(
  Survived ~., data = train.data, method = "svmLinear",
  trControl = trainControl("cv", number = 10),
  tuneGrid = expand.grid(C = model$bestTune)
)

predicted.classes <- model %>% predict(test.data)

confusionMatrix(factor(predicted.classes), factor(test.data$Survived), positive = '1')
```

Model with best Tune

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0  1
##           0 93 23
##           1 13 49
##
##           Accuracy : 0.7978
##           95% CI : (0.7312, 0.8541)
##           No Information Rate : 0.5955
##           P-Value [Acc > NIR] : 7.457e-09
##
##           Kappa : 0.5706
##
##           Mcnemar's Test P-Value : 0.1336
##
##           Sensitivity : 0.6806
##           Specificity : 0.8774
##           Pos Pred Value : 0.7903
##           Neg Pred Value : 0.8017
##           Prevalence : 0.4045
##           Detection Rate : 0.2753
##           Detection Prevalence : 0.3483
##           Balanced Accuracy : 0.7790
##
##           'Positive' Class : 1
##
```

With TuneLength = 10, radial basis kernel

```

set.seed(123)
model <- train(
  Survived ~.,
  data = train.data,
  method = "svmRadial",
  trControl = trainControl("cv", number = 10),
  # preProcess = c("center", "scale")
  # This will scale all the predictors in the model
  tuneLength = 10
)

```

```
model$bestTune
```

Best Tune

```
##          sigma C
## 3 0.1420266 1
```

```
predicted.classes <- model %>% predict(test.data)
```

Prediction

```

confusionMatrix(
  factor(predicted.classes),
  factor(test.data$Survived),
  positive = '1'
)

```

Confusion Matrix

```

## Confusion Matrix and Statistics
##
##          Reference
## Prediction  0   1
##          0 90 17
##          1 16 55
##
##          Accuracy : 0.8146
##          95% CI : (0.7496, 0.8688)
##    No Information Rate : 0.5955
##    P-Value [Acc > NIR] : 3.229e-10
##
##          Kappa : 0.6143

```

```
##
## McNemar's Test P-Value : 1
##
##      Sensitivity : 0.7639
##      Specificity : 0.8491
##      Pos Pred Value : 0.7746
##      Neg Pred Value : 0.8411
##      Prevalence : 0.4045
##      Detection Rate : 0.3090
##      Detection Prevalence : 0.3989
##      Balanced Accuracy : 0.8065
##
##      'Positive' Class : 1
##
```

TuneLength = 4, polynomial basis kernel

```
set.seed(123)
model <- train(
  Survived ~.,
  data = train.data,
  method = "svmPoly",
  trControl = trainControl("cv", number = 10),
  # preProcess = c("center", "scale")
  # This will scale all the predictors in the model
  tuneLength = 4
)
```

```
model$bestTune
```

Best Tune

```
##      degree scale      C
## 25         2    0.1 0.25
```

```
predicted.classes <- model %>% predict(test.data)
```

Prediction

```
confusionMatrix(
  factor(predicted.classes),
  factor(test.data$Survived),
  positive = '1'
)
```

Confusion Matrix

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  0   1
##           0 93 21
##           1 13 51
##
##           Accuracy : 0.809
##           95% CI : (0.7434, 0.8639)
##           No Information Rate : 0.5955
##           P-Value [Acc > NIR] : 9.523e-10
##
##           Kappa : 0.5963
##
##           Mcnemar's Test P-Value : 0.2299
##
##           Sensitivity : 0.7083
##           Specificity : 0.8774
##           Pos Pred Value : 0.7969
##           Neg Pred Value : 0.8158
##           Prevalence : 0.4045
##           Detection Rate : 0.2865
##           Detection Prevalence : 0.3596
##           Balanced Accuracy : 0.7928
##
##           'Positive' Class : 1
##
```

Result

Overall accuracy rate:

Linear: 0.7978

Radial basis kernel: 0.8146 (the best)

Polynomial kernel: 0.809